

Preparação para checagem

Grafos Bipartidos e 2-Coloração

Prof. Me. Luis Vinicius

UniAtenas

O que a prova pede?

Situação-problema

Uma instituição quer dividir estudantes em **dois grupos**. Alguns pares de estudantes têm **conflitos** e **não podem** ficar no mesmo grupo.

Sua tarefa:

- Modelar o problema como um **grafo não direcionado**
- Verificar se é possível fazer a divisão **respeitando todos os conflitos**
- Ou seja: verificar se o grafo é **bipartido**

Saída esperada (exata)

DIVISAO_POSSIVEL ou DIVISAO_IMPOSSIVEL

Modelagem do Problema

Elementos

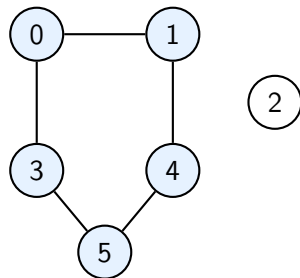
- **Vértice** = estudante
- **Aresta** = conflito entre dois estudantes
- Grafo **não direcionado**

Exemplo

Estudantes: 0, 1, 2, 3, 4, 5

Conflitos:

0-1, 0-3, 1-4, 3-5, 4-5



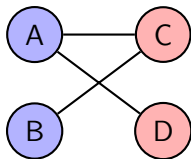
Vértice 2 isolado (sem conflitos)

O que é um Grafo Bipartido?

Definição

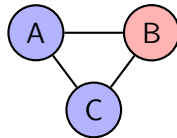
Um grafo é **bipartido** se seus vértices podem ser divididos em **dois conjuntos** (grupos) de modo que **toda aresta** conecte vértices de grupos **diferentes**.

BIPARTIDO



Todas as arestas ligam grupos diferentes

NÃO BIPARTIDO



Ciclo ímpar de tamanho 3

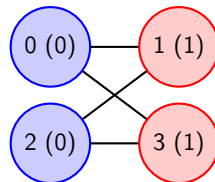
Teorema importante

Um grafo é bipartido se e somente se ele **não possui ciclos de comprimento ímpar**.

Exemplo 1: Grafo Bipartido (Divisão Possível)

- **Cenário:** 4 estudantes e 4 arestas formando um ciclo par.
- **Comando:** `./grupos 4 4`
- **Entrada:**
0 1
1 2
2 3
3 0
- **Saída:** `DIVISAO_POSSIVEL`

Visualização

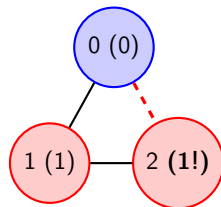


Cores alternadas sem conflito interno.

Exemplo 2: Grafo Não Bipartido (Divisão Impossível)

- **Cenário:** 3 estudantes formando um triângulo (ciclo ímpar).
- **Comando:** `./grupos 3 3`
- **Entrada:**
0 1
1 2
2 0
- **Saída:** `DIVISAO_IMPOSSIVEL`

Visualização



Colisão: Vértices 1 e 2 no mesmo Grupo 1.

Exemplo 3: Componentes Desconectados

- **Cenário:** 6 estudantes divididos em dois subgrafos isolados.

- **Comando:** `./grupos 6 4`

- **Entrada:**

0 1

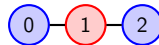
1 2

3 4

4 5

- **Saída:** `DIVISAO_POSSIVEL`

Visualização



O laço externo valida cada componente separadamente.

Estratégia: 2-Coloração com BFS

- 1 Inicializar todos os vértices como **não coloridos** (-1)
- 2 Escolher um vértice ainda não visitado e atribuí-lo ao **Grupo 0**
- 3 Executar **BFS** a partir desse vértice
- 4 Para cada vizinho: atribuir a **cor oposta**
 - Grupo 0 $\rightarrow$ Grupo 1
 - Grupo 1 $\rightarrow$ Grupo 0
- 5 Se encontrar aresta entre dois vértices da **mesma cor** $\rightarrow$ **impossível**
- 6 Se existirem outros componentes desconectados, **repetir** o processo
- 7 Se todos os componentes forem 2-coloridos sem conflito $\rightarrow$ **possível**

Lista de Adjacência (não direcionada)

O que você precisa completar

```
adicionar_aresta(Grafo *g, int u, int v)
```

Importante: como o grafo é **não direcionado**, a aresta deve aparecer nos **dois sentidos**:

- Inserir v na lista de adjacência de u
- Inserir u na lista de adjacência de v

```
void adicionar_aresta(Grafo *g, int u, int v) {  
    // 1. Cria no com vertice v e liga em g->adj[u]  
    No *novo1 = criar_no(v);  
    novo1->prox = g->adj[u].inicio;  
    g->adj[u].inicio = novo1;  
  
    // 2. Cria no com vertice u e liga em g->adj[v]  
    No *novo2 = criar_no(u);  
    novo2->prox = g->adj[v].inicio;  
    g->adj[v].inicio = novo2;  
}
```

BFS + 2-Coloração – Ideia Central

- Vetor de cores:
 - -1 = não colorido
 - 0 = Grupo 0
 - 1 = Grupo 1
- Laço externo percorre **todos** os vértices
- Se encontrar um não colorido, inicia nova BFS
- Dentro da BFS:
 - Tira da fila
 - Percorre vizinhos
 - Se vizinho não colorido → colorir com cor oposta e enfileirar
 - Se vizinho já tem a **mesma cor** → retorna falso

Cor oposta

$$\text{cor_oposta} = 1 - \text{cor}$$

0 vira 1

1 vira 0

Atenção

Não esqueça dos **componentes desconectados!**

Entrada: como os conflitos são informados?

Exemplo

```
./grupos 4 3
```

```
0 1
```

```
1 2
```

```
2 3
```

- 4 estudantes $\rightarrow$ vértices 0, 1, 2, 3
- 3 conflitos $\rightarrow$ três arestas
- Cada linha $u\ v$ significa: u e v têm conflito

Da entrada para a matriz de adjacência

Considere:

0 1

1 2

2 3

Cada conflito recebe valor 1 na matriz.

$$A[i][j] = \begin{cases} 1 & \text{se existe conflito entre } i \text{ e } j \\ 0 & \text{caso contrário} \end{cases}$$

Regra

Para cada conflito u v :

$$A[u][v] = 1 \quad A[v][u] = 1$$

Como o grafo é **não direcionado**, a matriz é simétrica.

Exemplo: matriz de adjacência

Entrada:

```
./grupos 4 3
```

```
0 1
```

```
1 2
```

```
2 3
```

	0	1	2	3
0	0	1	0	0
1	1	0	1	0
2	0	1	0	1
3	0	0	1	0

- Linha = vértice atual
- Coluna = possível vizinho
- 1 = existe conflito
- 0 = não existe conflito

Como ler a matriz?

Considere:

	0	1	2	3
0	0	1	0	0
1	1	0	1	0
2	0	1	0	1
3	0	0	1	0

Exemplos

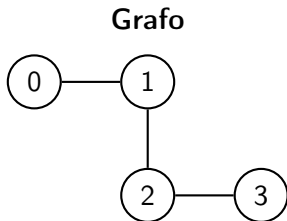
$$A[0][1] = 1$$

⇒ existe conflito entre 0 e 1.

$$A[0][2] = 0$$

⇒ não existe conflito entre 0 e 2.

Grafo × Matriz de Adjacência



Matriz

	0	1	2	3
0	0	1	0	0
1	1	0	1	0
2	0	1	0	1
3	0	0	1	0

Mesma informação

A matriz é apenas outra forma de representar os mesmos **conflitos entre os estudantes**.

Como a BFS usa a matriz?

Para processar um vértice u , percorremos toda a sua linha:

$$A[u][0], A[u][1], \dots, A[u][V - 1]$$

Exemplo: processando o vértice 1

A linha do vértice 1 é:

$$[1, 0, 1, 0]$$

Portanto:

- $A[1][0] = 1 \rightarrow$ existe vizinho 0
- $A[1][1] = 0 \rightarrow$ não existe aresta para 1
- $A[1][2] = 1 \rightarrow$ existe vizinho 2
- $A[1][3] = 0 \rightarrow$ não existe aresta para 3

Assim, os vizinhos de 1 são:

Matriz + BFS + 2-Coloração

A matriz informa **quem tem conflito com quem**.

O vetor de cores informa **em qual grupo cada estudante está**.

$$\text{cor}[v] = \begin{cases} -1 & \text{não colorido} \\ 0 & \text{Grupo 0} \\ 1 & \text{Grupo 1} \end{cases}$$

Durante a BFS

Se:

$$A[u][v] = 1$$

então u e v têm conflito.

Portanto, eles precisam estar em grupos diferentes:

$$\text{cor}[v] = 1 - \text{cor}[u]$$

Quando a divisão é impossível?

Suponha que:

$$A[u][v] = 1$$

e os dois vértices já tenham a mesma cor:

$$\text{cor}[u] = \text{cor}[v]$$

Então existe um conflito dentro do mesmo grupo.

Condição de impossibilidade

$$A[u][v] = 1 \quad \wedge \quad \text{cor}[u] = \text{cor}[v]$$

$$\Rightarrow \boxed{\text{DIVISAO_IMPOSSIVEL}}$$

Exemplo: conflito detectado pela matriz

Entrada:

./grupos 3 3

0 1

1 2

2 0

Matriz:

	0	1	2
0	0	1	1
1	1	0	1
2	1	1	0

Começamos:

$cor[0] = 0$

Como 0 tem conflito com 1 e 2:

Implementação usando matriz

Se a prova utilizasse matriz de adjacência, poderíamos ter:

```
#define MAX 100

int adj[MAX][MAX];
int cor[MAX];
```

Para adicionar um conflito:

```
void adicionar_aresta(int u, int v) {
    adj[u][v] = 1;
    adj[v][u] = 1;
}
```

Para verificar os vizinhos de u :

```
for (int v = 0; v < n; v++) {
    if (adj[u][v]) {
        // v é vizinho de u
    }
}
```

Matriz de Adjacência $\times$ Lista de Adjacência

	Matriz	Lista
Representação	$V \times V$	Vizinhos de cada vértice
Testar $u - v$	$O(1)$	$O(\deg(u))$
Percorrer vizinhos	$O(V)$	$O(\deg(u))$
Espaço	$O(V^2)$	$O(V + E)$

Para esta prova

O enunciado/material trabalha com **lista de adjacência**.

A matriz é útil para entender a representação dos conflitos, mas a implementação fornecida deve ser feita com a estrutura esperada pela prova.

Implementação da função bipartido_bfs

```
int bipartido_bfs(Grafo *g) {
    int *cor = malloc(g->n * sizeof(int));
    for (int i = 0; i < g->n; i++) cor[i] = -1; // nao colorido

    for (int s = 0; s < g->n; s++) { // todos os componentes
        if (cor[s] != -1) continue;

        Fila *f = criar_fila(g->n);
        cor[s] = 0; // Grupo 0
        enfileirar(f, s);

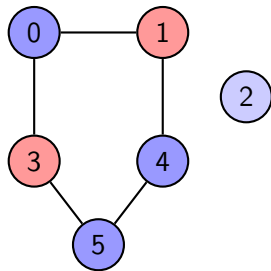
        while (!fila_vazia(f)) {
            int u = desenfileirar(f);
            for (No *p = g->adj[u].inicio; p != NULL; p = p->prox) {
                int v = p->vertice;
                if (cor[v] == -1) { // ainda nao colorido
                    cor[v] = 1 - cor[u]; // cor oposta
                    enfileirar(f, v);
                } else if (cor[v] == cor[u]) { // mesmo grupo!
                    free(cor); liberar_fila(f);
                    return 0; // IMPOSSIVEL
                }
            }
        }
        liberar_fila(f);
    }
    free(cor);
    return 1; // POSSIVEL
}
```

Exemplo 1 – Grafo Bipartido

Entrada: ./grupos 6 5

Conflitos: 0-1, 0-3, 1-4, 3-5, 4-5

- 1 Começa no 0 $\rightarrow$ cor 0
- 2 Vizinhos de 0 (1 e 3) $\rightarrow$ cor 1
- 3 Vizinhos de 1: 4 $\rightarrow$ cor 0
- 4 Vizinhos de 3: 5 $\rightarrow$ cor 0
- 5 Vizinhos de 4: 5 já tem cor 0 (mesma de 4?)
Não! 4 tem cor 0 e 5 tem cor 0?
Espere... 4 e 5 estão conectados!



Cuidado no exemplo

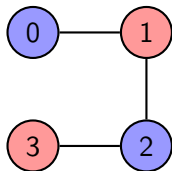
Neste grafo existe o ciclo 1-4-5-3-0-1 de tamanho 5 (ímpar)!

Portanto a saída correta é `DIVISAO_IMPOSSIVEL`.

Exemplo Correto – Grafo Bipartido

Entrada sugerida para Teste 1:

```
./grupos 4 3  
0 1  
1 2  
2 3
```



Grupo 0: {0, 2}

Grupo 1: {1, 3}

Resultado

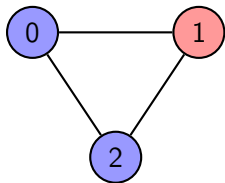
DIVISAO_POSSIVEL

Não há aresta entre vértices do mesmo grupo.
Não há ciclo ímpar.

Exemplo – Grafo Não Bipartido (ciclo ímpar)

Entrada sugerida para Teste 2:

```
./grupos 3 3  
0 1  
1 2  
2 0
```



Ciclo de tamanho 3 (ímpar)

Resultado

DIVISAO_IMPOSSIVEL

Quando a BFS chega no último vértice, ele já tem a mesma cor de um vizinho → conflito detectado.

Teste 3 – Componentes Desconectados

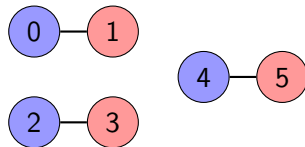
Entrada sugerida:

```
./grupos 6 3  
0 1  
2 3  
4 5
```

Três componentes separados:

- {0-1}
- {2-3}
- {4-5}

O laço externo da função deve iniciar uma nova BFS em cada componente não visitado.



DIVISAO_POSSIVEL

Complexidade de Tempo

$$O(V + E)$$

onde V = número de vértices (estudantes) e E = número de arestas (conflitos).

Cada vértice é enfileirado/desenfileirado no máximo uma vez e cada aresta é examinada no máximo duas vezes (uma em cada direção).

Complexidade de Espaço

$$O(V + E)$$

- Lista de adjacência: $O(V + E)$
- Vetor de cores: $O(V)$

Teorema

Um grafo é bipartido $\iff$ ele não contém ciclos de comprimento ímpar.

Por quê a 2-coloração detecta isso?

- Em um ciclo de tamanho ímpar, é impossível colorir alternando duas cores sem que dois vértices adjacentes fiquem com a mesma cor.
- A BFS (ou DFS) encontra essa inconsistência quando tenta colorir um vértice que já foi colorido com a cor “errada”.

Exemplos de ciclos ímpares

Triângulo (3), pentágono (5), heptágono (7)...

Pontos de Atenção na Prova

- **Saída exata:** apenas `DIVISAO_POSSIVEL` ou `DIVISAO_IMPOSSIVEL` (sem espaços extras, sem mensagens)
- Complete **apenas** os trechos marcados com `COMPLETE`
- Representação **não direcionada** (duas inserções na lista)
- Tratar **todos os componentes** (laço externo)
- Liberar memória da fila e do vetor de cores
- Testar os **três casos obrigatórios**:
 - 1 Grafo bipartido
 - 2 Grafo com ciclo ímpar
 - 3 Componentes desconectados

Critérios de avaliação (resumo)

Lista de adjacência + BFS + cores + componentes + testes + explicação + complexidade + ciclos ímpares

Checklist Final antes de entregar

- ✓ adicionar_aresta insere nos **dois** lados
- ✓ Vetor de cores inicializado com -1
- ✓ Laço externo cobre todos os componentes
- ✓ Cor oposta calculada como $1 - \text{cor}[u]$
- ✓ Detecção de conflito: $\text{cor}[v] == \text{cor}[u]$
- ✓ Retorna 1 (possível) ou 0 (impossível)
- ✓ Saída **exatamente** como especificado
- ✓ Pelo menos 3 testes realizados
- ✓ Consegue explicar a lógica oralmente

Boa prova!